

Accenture Assessment — Topics & Learning Roadmap

Derived from pattern analysis of 8 past-paper PDFs, then extended with the surrounding material those papers imply but do not happen to cover.

How to read this

The eight source PDFs contain roughly 250 questions. Stripped of duplicates and phrasing, they test about **55 distinct concepts**. This document lists those concepts in the order you should learn them, and then adds the adjacent topics that the same syllabus almost certainly draws from.

Every topic carries a priority tag:

Tag	Meaning
CORE	Appeared 2+ times across the papers. Non-negotiable.
SEEN	Appeared once. Learn it.
EXTEND	Did not appear, but sits inside the same syllabus block and is commonly asked. Learn it after the CORE and SEEN items.

Work top to bottom. Each level assumes the one above it.

What the papers actually repeat

Before the roadmap, here is the pattern analysis — the twelve concepts that appeared most often across the eight PDFs. If you only have three days, this table *is* your syllabus.

Concept	Times seen	Where
VPN type selection (Remote Access / Site-to-Site / IPSec / hybrid)	6	Networking, Actual Paper
Containers vs Virtual Machines, and kernel-level container isolation	6	Networking, Actual Paper, CS
Cloud service & deployment models (IaaS/PaaS/SaaS, Public/Private/Hybrid)	5	Networking, Actual Paper
HTTP methods, REST resource naming, status codes	5	CS, Actual Paper
Program Counter & context switching	3	Actual Paper, Common Apps
Accumulator → memory = store / memory write	3	Actual Paper, Common Apps, CS
Continuous Deployment vs CI vs Continuous Testing	3	CS, Actual Paper
Device controllers (EIDE vs SATA)	3	Actual Paper, Common Apps
"Second largest at even/odd positions" coding problem	3	Coding
OSI layer identification (Physical / Network / Transport)	3	Networking, Actual Paper
Bitwise operators inside pseudocode traces	6+	Pseudo Code
Root-cause-first troubleshooting (memory leak, latency)	3	CS, Actual Paper

The meta-pattern worth internalising: whenever a scenario question offers "throw more resources at it" versus "diagnose the cause", the answer is always diagnose. Whenever it offers a generic answer versus a specific named mechanism, the answer is the specific one.

Level 0 — Prerequisites

If any of these is shaky, fix it before anything else. Half a day, total.

Topic	Priority	What to know
Number bases	CORE	Binary ↔ decimal ↔ hex conversion by hand. Powers of 2 up to 2^{10} .
Bitwise operators	CORE	<code>&</code> , <code>&#124;</code> (OR), <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code> . Be able to compute <code>29 & 4</code> mentally by writing both in binary. Six pseudocode questions hinge on this alone.
Operator precedence & evaluation order	CORE	Arithmetic before comparison before logical. But note: the Binary Operations coding problem explicitly says <i>ignore</i> precedence — read the question.
0-indexed vs 1-indexed	CORE	"Second element" means index 1. Asked directly, and it silently breaks half the coding problems.
Integer division & modulo	CORE	<code>7/2 = 3</code> in integer arithmetic; <code>%</code> for digit extraction and parity.
Time & space complexity	EXTEND	Big-O of the common patterns. Not asked as theory, but Tier-3 coding problems fail on timeouts without it.

Level 1 — Programming Logic & Pseudocode

The highest-yield section in the entire assessment: it is fully deterministic, requires no memorisation, and you can reach 100% accuracy with a week of tracing.

Topic	Priority	What to know
Hand-tracing variable state	CORE	Write a table of variables down the page, one row per line of code. Never trace in your head.
Nested if / else-if chains	CORE	The dominant question shape. Usually the <i>outer</i> condition is false and nothing executes — check it first and you may be done.
Loop tracing	CORE	<code>for(each c from 3 to 4)</code> runs twice . Range-inclusive loop bounds are a deliberate trap.
Functions & return values	CORE	A function that returns a constant ignores every field you set beforehand.
Recursion & base cases	CORE	Recognise factorial, Fibonacci and sum-of-n patterns from their base case alone.
2-D array indexing	CORE	<code>arr[row][col]</code> . Initialisation <code>{{0,1},{4,1}}</code> means row 0 is <code>{0,1}</code> .
Regex basics	SEEN	Character classes <code>[a-zA-Z0-9]</code> , quantifiers <code>{6}</code> , anchors <code>^</code> , <code>\$</code> , <code>\d</code> , <code>\w</code> , <code>\s</code> . One question, but it is free if you know it.
Pass by value vs reference	EXTEND	Why modifying an int inside a function doesn't stick but modifying an array does.
Short-circuit evaluation	EXTEND	<code>&&</code> stops at the first false, <code>&#124;&#124;</code> at the first true. Common in trickier traces.

Level 2 — Data Structures & Algorithms (the coding round)

68 problems in the source PDF collapse into about 12 reusable patterns. Learn the patterns, not the 68.

2.1 Patterns that actually appear

Pattern	Priority	Problems it solves
Frequency counting (HashMap / 26-array)	CORE	Anagram, most frequent vowel, count occurrences, rhyme words, autobiographical number, replace frequent characters
Two pointers	CORE	Reverse string, merge sorted arrays, palindrome, pair-sum-max-product, rotate by k
Sliding window (fixed & variable)	CORE	Longest substring without repeats, max favourite song, chocolate distribution
Prefix / running sum	CORE	Equilibrium sum, rat count house, sum at even indices
Single-pass max/min tracking	CORE	Maximum with index, second largest, elevation point, max successive difference
Digit manipulation	CORE	Count carry, encode number, googly prime, palindrome numbers, sum of divisors
Bit manipulation	CORE	Sum of binary digits (popcount), magical numbers, rearrangement of bits, max exponent of 2
Sorting + scan	CORE	Chocolate distribution, max successive difference, large-small sum, product of smallest pair
In-place matrix marking	SEEN	Set matrix zeroes
Linked list slow/fast pointers	SEEN	Linked list palindrome
String parsing & tokenising	CORE	First K words, reverse words, length of last word, string decoder, move hyphens
Greedy simulation	SEEN	Bulb switch

2.2 Data structures to be fluent in

Topic	Priority	Notes
Arrays & 2-D arrays	CORE	Traversal, in-place modification, index arithmetic
Strings & StringBuilder	CORE	Immutability; never <code>+=</code> in a loop
HashMap / HashSet	CORE	The workhorse. Know <code>getOrDefault</code> , iteration over entries
Linked list	SEEN	Traversal, reversal, slow/fast pointers, insertion/deletion
Stack & Queue	EXTEND	Balanced brackets, next greater element, BFS — common in the harder variants of this paper
Sorting algorithms	EXTEND	Know bubble/selection/insertion by hand, merge and quick conceptually, and their complexities
Binary search	EXTEND	On a sorted array and on an answer range. Turns several Tier-2 problems from $O(n)$ into $O(\log n)$
Recursion & backtracking	EXTEND	Permutations, subsets, N-queens shape problems
Trees & BST basics	EXTEND	Traversals, height, search. Not in these papers, but standard in the interview that follows

2.3 Edge cases (this is what separates 60% from 100%)

Null input · empty array · single element · two elements · all elements identical · negative numbers · duplicates of the maximum · integer overflow · the exact return value the Notes block demands (-1 vs 0) · output format with no prompts and no trailing whitespace.

Level 3 — Databases & SQL

Topic	Priority	What to know
SELECT / FROM / WHERE clause order	CORE	SELECT always precedes FROM. Two questions test only this.
LIKE and wildcards	CORE	'a%' starts with, '%a' ends with, '%a%' anywhere, _ one character. Asked four times.
JOINS	CORE	INNER (matches only), LEFT (all left rows + NULLs), RIGHT, FULL. The ON clause, never IN. LEFT JOIN is asked explicitly.
Multi-table joins	CORE	Chaining 2-3 joins along foreign keys is the standard query-writing question shape.
Self-join	SEEN	The hardest query in the set (users joined to users via a contacts table).
Aliases (AS)	CORE	The grader checks column names. AS "Employee ID" with quotes when the alias has a space.
ORDER BY / ASC / DESC	CORE	
GROUP BY + aggregates	CORE	COUNT, SUM, AVG, MAX, MIN. Every non-aggregated selected column must appear in GROUP BY.
HAVING vs WHERE	CORE	WHERE filters rows before grouping; HAVING filters groups after.
DISTINCT	CORE	
String functions	CORE	CONCAT(a, ' ', b), LENGTH, REPLACE, UPPER, LOWER, SUBSTRING, TRIM. The LENGTH-LENGTH(REPLACE) trick for counting characters is asked directly.
Date functions	SEEN	EXTRACT(YEAR FROM col), date comparison with 'YYYY-MM-DD' strings, NOW().
Comparison operators	CORE	<> / != for "other than", IN (...) for membership, BETWEEN.
REGEXP	SEEN	NAME REGEXP '[0-9]' and the LIKE-chain fallback.
UPDATE / DELETE with WHERE	CORE	
Indexing	CORE	Why an index on a searched column beats a full scan on 5M rows.
Triggers	SEEN	AFTER DELETE ... FOR EACH ROW, and the OLD vs NEW row references.
Primary key properties	CORE	Unique, non-null, immutable.
Transaction anomalies	SEEN	Lost update, dirty read, non-repeatable read, phantom read.
ACID properties	EXTEND	Atomicity, Consistency, Isolation, Durability — the natural follow-up to the anomalies question.
Normalisation (1NF-3NF, BCNF)	EXTEND	Classic campus-test material entirely absent from these papers, which makes it likely in another set.
Subqueries & nested SELECT	EXTEND	Including correlated subqueries and EXISTS.
Views, stored procedures	EXTEND	
Foreign keys & referential integrity	EXTEND	
SQL vs NoSQL	EXTEND	When each is appropriate.

Level 4 — Object-Oriented Programming

Topic	Priority	What to know
The four pillars	CORE	Encapsulation, Abstraction, Inheritance, Polymorphism — with a one-line example of each.
Overloading vs overriding	CORE	Same name + different parameters, same class = overloading = static/compile-time polymorphism. Same signature in a subclass = overriding = dynamic/runtime . Asked twice.
is-a vs has-a	CORE	<code>extends</code> → is-a (inheritance). A field of another class → has-a (composition/association).
Access modifiers	CORE	<code>private</code> members are not inherited — the basis of an entire question. Know private / default / protected / public.
Static vs instance members	SEEN	A method returning a constant ignores instance state.
Constructors	EXTEND	Default, parameterised, chaining, the order of initialisation.
Abstract classes vs interfaces	EXTEND	The single most common OOP interview question, absent here.
<code>final</code> , <code>this</code> , <code>super</code>	EXTEND	
Exception handling	EXTEND	try/catch/finally, checked vs unchecked.
Collections framework	EXTEND	List/Set/Map, ArrayList vs LinkedList, HashMap vs TreeMap.

Level 5 — Operating Systems

Topic	Priority	What to know
Process creation	CORE	<code>fork()</code> duplicates, <code>exec()</code> replaces the image, <code>wait()</code> blocks the parent. Asked twice.
Context switching	CORE	What is saved (PCB: Program Counter , registers, stack pointer, state). The PC question appears three times.
OS core functions	CORE	Process management, memory management, device management , file management, access control . Know which one a scenario is pointing at.
Access control & OS security	CORE	Permissions, authentication, the principle of least privilege.
File-management subsystem	SEEN	Permissions and access control as the organising function, above compression/encryption/backup.
Process vs thread	EXTEND	
CPU scheduling	EXTEND	FCFS, SJF, Round Robin, priority — with the ability to compute waiting and turnaround time. Very commonly asked, absent here.
Deadlock	EXTEND	The four Coffman conditions; prevention, avoidance (Banker's), detection.
Memory management	EXTEND	Paging, segmentation, virtual memory, page faults, LRU/FIFO replacement.
Synchronisation	EXTEND	Race conditions, critical sections, mutexes, semaphores, producer-consumer.
Linux command line	EXTEND	<code>ls cd grep find chmod ps kill top df</code> — expected of any developer hire.

Level 6 — Computer Architecture

Topic	Priority	What to know
Accumulator ↔ memory operations	CORE	CPU → memory = store / memory write ; memory → CPU = load / memory read. Three occurrences.
CPU registers	CORE	PC (next instruction address), IR (current instruction), MAR, MDR, accumulator, stack pointer.
System bus	SEEN	Data bus + address bus + control bus as the pathway between CPU, memory and I/O.
Device controllers	CORE	EIDE (enhanced IDE/ATA, built-in cache, storage) vs SATA (serial, high-speed buffers) vs USB vs Ethernet. Three occurrences, and the distractors are near-anagrams.
I/O handling	SEEN	Host controller → device controller messaging.
Instruction cycle	EXTEND	Fetch → decode → execute → store, and where the PC increments.
Memory hierarchy	EXTEND	Registers → cache (L1/L2/L3) → RAM → disk, by speed, cost and size.
Cache basics	EXTEND	Hit/miss, locality of reference, write-through vs write-back.
Interrupts vs polling vs DMA	EXTEND	
Pipelining	EXTEND	

Level 7 — Web Technologies & APIs

Topic	Priority	What to know
HTTP methods	CORE	GET (read, safe, idempotent), POST (create/submit), PUT (replace), PATCH (partial), DELETE. Asked four times.
HTTP status codes	CORE	200, 201, 301, 400, 401, 403, 404 , 500, 502, 503.
REST design principles	CORE	Noun-based resource paths, identifier in the path, verb carried by the HTTP method, no <code>/getX</code> or <code>/details</code> suffixes.
API versioning	SEEN	URL-path versioning <code>/api/v2/resource</code> as the recommended strategy for breaking changes.
CORS & preflight	SEEN	Why browsers send an <code>OPTIONS</code> request before non-simple cross-origin calls.
DOM manipulation	CORE	<code>getAttribute</code> returns the literal attribute string; <code>createAttribute</code> + <code>setAttributeNode</code> .
JavaScript array methods	SEEN	<code>map</code> (transform, same length), <code>filter</code> (subset), <code>reduce</code> (collapse), <code>forEach</code> (no return).
HTML5 elements	SEEN	<code><canvas></code> and <code><audio></code> / <code><video></code> fallback content; the <code>autoplay</code> attribute; multiple <code><source></code> children.
DOM events	SEEN	<code>onclick</code> , <code>onload</code> , <code>onabort</code> , <code>onchange</code> , <code>onsubmit</code> .
CSS	SEEN	<code>border-radius</code> , <code>transition</code> and its easing, <code>:hover</code> .
Request/response cycle	EXTEND	What actually happens between typing a URL and seeing a page — the classic follow-up interview question.
Cookies, sessions, JWT	EXTEND	
SOAP vs REST, JSON vs XML	EXTEND	
Synchronous vs asynchronous, promises, async/await	EXTEND	
<code>var</code> / <code>let</code> / <code>const</code> , <code>==</code> vs <code>===</code> , hoisting	EXTEND	Standard JS screening material.

Level 8 — Computer Networks

Topic	Priority	What to know
OSI model, all 7 layers	CORE	Which device, protocol and data unit lives at each layer. Physical, Network and Transport are each asked directly.
TCP vs UDP	CORE	Reliable/ordered/handshake vs fast/connectionless/lossy. DNS uses UDP — asked directly.
TCP three-way handshake	SEEN	SYN → SYN-ACK → ACK.
Email protocols	CORE	SMTP sends; POP3 downloads and deletes; IMAP syncs and retains.
Network topologies	CORE	Ring (one failure breaks the loop), star, bus, mesh — their failure behaviour is the question.
MAC filtering	SEEN	Access control by hardware address.
Wi-Fi security	CORE	WEP < WPA < WPA2 < WPA3 . WPA3-Personal uses SAE; WPA3-Enterprise uses 802.1X/EAP.
VPN types	CORE	Remote Access (one user) / Site-to-Site (two networks) / IPSec (protocol-level) / SSL VPN. Six occurrences — the single most-repeated concept in the papers.
IPSec	SEEN	ESP = encryption + authentication; AH = authentication only; IKE = key exchange; transport vs tunnel mode.
TCP/IP model	EXTEND	The 4-layer model and how it maps onto OSI.
IP addressing & subnetting	EXTEND	IPv4 classes, CIDR, subnet masks, public vs private ranges, NAT, IPv6 basics. A standard campus-test topic entirely absent here.
DNS & DHCP	EXTEND	Resolution flow; how a device gets its address.
Routers vs switches vs hubs vs gateways	EXTEND	
Common port numbers	EXTEND	20/21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 80 HTTP, 110 POP3, 143 IMAP, 443 HTTPS, 3306 MySQL.
Latency, bandwidth, throughput, jitter	EXTEND	

Level 9 — Security

Topic	Priority	What to know
Encryption vs hashing vs encoding	CORE	Reversible with a key / one-way / reversible without a key. Also distinguish authentication (who) from authorization (what).
Symmetric vs asymmetric	CORE	Shared key vs public/private pair. The chosen-plaintext question assumes you know the symmetric setup.
SSL/TLS	CORE	Securing data in transit; what turns HTTP into HTTPS.
Attack taxonomy	CORE	Source routing, routing table poisoning , MITM, DDoS, IP spoofing, DNS spoofing, ARP poisoning, packet sniffing, tiny fragment attacks. Be able to pick the <i>specific</i> mechanism over the general category.
Cryptanalysis attacks	SEEN	Chosen-plaintext, known-plaintext, ciphertext-only, brute force.
Firewall types	CORE	Packet-filtering (stateless) → stateful inspection → circuit-level gateway → application/proxy. Match the type to the capability described.
DDoS mitigation	SEEN	Filtering bot floods while preserving legitimate traffic.
Network segmentation	SEEN	Its security purpose is limiting lateral movement, not performance.
MFA	CORE	Know (password) / have (token) / are (biometric).
IT governance vs operations	CORE	Policy ownership stays in-house; operational work can be outsourced. Appears twice in different clothing.
Shared responsibility model	SEEN	Provider secures <i>of</i> the cloud; customer secures <i>in</i> the cloud.
Data in transit vs at rest vs in use	EXTEND	
OWASP Top 10	EXTEND	Especially SQL injection and XSS — the natural companions to the SQL section.
Principle of least privilege, zero trust	EXTEND	
Digital signatures, certificates, PKI	EXTEND	

Level 10 — Cloud & DevOps

Topic	Priority	What to know
Service models	CORE	IaaS (you manage OS + app) / PaaS (app only) / SaaS (nothing). Match a scenario to the right layer.
Deployment models	CORE	Public / Private (most secure for sensitive data) / Hybrid (on-prem + public) / Community.
Cloud storage types	CORE	Object (scales dynamically, unstructured, S3-style) / Block (volumes) / File (shared filesystem).
Elasticity, scalability, resource pooling	CORE	Vertical vs horizontal scaling; pooling multi-vendor storage into one virtual pool.
Auto-scaling & load balancing	CORE	The paired answer to every traffic-spike question.
Cloud cost models	SEEN	On-demand vs reserved instances vs spot.
Containers vs VMs	CORE	Shared kernel vs own guest OS. Six occurrences across the papers.
Container security & isolation	CORE	Kernel-enforced resource and device isolation. Three occurrences, near-identical wording.
Container troubleshooting	CORE	Memory leak → analyse the code, not the limit. Latency → analyse logs and network first.
CI / CD / Continuous Deployment / Continuous Testing	CORE	"Automatically deployed to the live environment" = Continuous Deployment . Three occurrences.
Deployment strategies	SEEN	Rolling updates with health checks ; also know blue-green and canary.
Hybrid connectivity	CORE	VPN or a dedicated link (Direct Connect / ExpressRoute) between on-prem and cloud.
Data abstraction layer	SEEN	Unifying disparate sources during a legacy-to-cloud migration.
Docker & Kubernetes basics	EXTEND	Image vs container, Dockerfile, pod, deployment, service. Implied by every container question here.
Infrastructure as Code	EXTEND	Terraform / CloudFormation concepts.
Monitoring & observability	EXTEND	Logs, metrics, traces.
Microservices vs monolith	EXTEND	
Serverless	EXTEND	Appears as a distractor twice — know why it is <i>not</i> the isolation answer.

Level 11 — Software Engineering Process

Barely tested in these papers, but it is standard Accenture interview and assessment material and it frames the DevOps questions.

Topic	Priority	What to know
Agile & Scrum	SEEN	Sprints, standups, backlog, retrospective; roles (PO, SM, team). "Agile development approach" frames the CD question.
SDLC models	EXTEND	Waterfall vs Agile vs V-model vs Spiral, and when each fits.
Version control / Git	EXTEND	clone, branch, merge, rebase, pull request, merge conflicts.
Testing	EXTEND	Unit / integration / system / UAT; black-box vs white-box; regression; smoke.
Code review & clean code	EXTEND	
Design patterns	EXTEND	Singleton, Factory, Observer, MVC at a conceptual level.

Level 12 — MS Office / Common Applications

Do not skip this. It is the easiest section in the paper and the one most engineering candidates lose marks on.

Topic	Priority	What to know
Excel: PivotTables	CORE	Calculated Fields (new formula field) vs Calculated Items vs Slicers vs Grouping.
Excel: lookup & dynamic arrays	CORE	FILTER (all matches, Excel 365), VLOOKUP , INDEX + MATCH , XLOOKUP . Evaluate INDEX/MATCH inside-out .
Excel: range notation	CORE	B8:B10 = a range; colon vs comma.
Excel: common functions	EXTEND	SUM, AVERAGE, COUNT/COUNTA/COUNTIF, SUMIF, IF, nested IF, CONCATENATE/TEXTJOIN, absolute vs relative references (\$A\$1).
Word: paragraph formatting	CORE	Keep with next and Keep lines together.
Word: file formats	CORE	.docx vs .dotx (template) — and the same t -means-template rule for .xltx and .potx .
Word: shortcuts	SEEN	Alt+Shift+D date, Alt+Shift+T time, Shift+F5 last edit location.
Word: features	SEEN	Mini Translator, AutoRecover (auto-save), Track Changes, Mail Merge, Styles.
PowerPoint: slide show setup	CORE	Kiosk mode (Browsed at a kiosk) for the strictest navigation control.
PowerPoint: animation	CORE	Morph transition for progression <i>across</i> slides; By Paragraph effect option for bullets <i>within</i> one text box. Both are asked, and they are easy to confuse.
PowerPoint: other	EXTEND	SmartArt, Slide Master, Presenter View, Slide Zoom, section breaks.

Level 13 — Game-Based / Cognitive Round

Unrevisable as content; entirely trainable as technique.

Topic	Priority	What to practise
Number ordering under time pressure	CORE	Fraction → decimal conversion at speed. Know 1/3=0.33 , 2/3=0.67 , 1/6=0.167 , 1/7≈0.143 , 1/8=0.125 , 1/9=0.111 by heart. Expect deliberate ties.
Grid pathfinding with hidden walls	CORE	Maintain a mental map; never re-test a known wall; prefer moves that close both the row and column gap.
Tile rotation / path building	SEEN	Solve backwards from the exit; count rotations before committing.
Sequenced-objective planning	SEEN	Plan the full route before the first move when a wrong order resets progress.
Spatial working memory	EXTEND	Practise with any grid-memory or dual-n-back style app for 10 minutes a day. The measured trait is working-memory capacity, and it does improve with practice.
Aptitude fundamentals	EXTEND	Percentages, ratios, averages, time-speed-distance, permutation-combination, probability, series completion, syllogisms, blood relations, coding-decoding. Accenture's cognitive section draws from this pool.

Also worth knowing

Topics absent from these eight PDFs but that regularly appear in the same assessment family and in the interview that follows:

- **Generative AI & LLM basics** — Accenture now asks about this in both technical rounds and interviews. Know what a model, prompt, token, hallucination and RAG are, and have an honest opinion about where AI helps and where it does not.
- **Cognitive/aptitude section** — quantitative, logical reasoning, verbal ability, and the "Common Applications" section you have already seen.
- **Communication assessment** — Accenture runs a separate spoken/written English module. Practise reading aloud and short structured written answers.
- **Data structures beyond arrays** — trees, graphs, heaps, tries. Not in this paper set; standard in the technical interview.
- **DBMS theory** — normalisation, ACID, indexing internals, transactions. The papers test SQL syntax but never the theory, which suggests it sits in a different section.
- **Number systems & digital logic** — gates, K-maps, adders. Commonly bundled with the computer-architecture section.
- **Your own projects** — the single highest-return preparation item for the interview. For each project be able to state, in two minutes: the problem, your specific contribution, the hardest technical decision and why you made it, and what you would do

differently.

A 15-day schedule

Days	Morning (2h)	Evening (1.5h)	Coding (1h, every day)
1-2	Level 0 prerequisites + Level 1 pseudocode	Trace all 14 pseudocode questions by hand	Tier 1: problems C1-C12
3-4	Level 3 SQL concepts + query writing	Write all 30 queries by hand, no copy-paste	Tier 1: C13-C25
5-6	Level 2 DSA patterns	Re-solve the 3 patterns you were slowest at	Tier 2: C26-C40
7	Level 4 OOP + Level 5 OS	Flashcards on overloading/overriding, fork/exec, PC	Tier 2: C41-C53
8	Level 6 Architecture + Level 7 Web/API	HTTP status codes + REST naming drill	Tier 3: C54-C58
9-10	Level 8 Networks + Level 9 Security	Build the VPN table and the attack taxonomy from memory	Tier 3: C59-C64
11-12	Level 10 Cloud & DevOps	Containers-vs-VMs table and CI/CD ladder from memory	Tier 3: C65-C68
13	Level 12 MS Office	Practise INDEX/MATCH and FILTER in a real spreadsheet	Re-solve 5 random Tier-2 problems, timed
14	Level 13 game round + aptitude	Number-ordering drills, 20 in 10 minutes	Re-solve 5 random Tier-3 problems, timed
15	Full mock: 1 hour mixed MCQ + 2 coding problems, timed	Review only the errata list and your own wrong answers	—

If you have fewer than 15 days: do the frequency table at the top of this document, all 14 pseudocode questions, all 30 SQL queries, Tier 1 and Tier 2 coding, and the MS Office section. That is roughly 70% of the marks for 40% of the work.

Companion document: Consolidated Practice Question Bank — every question referenced here, in the same order, with answers and worked explanations.